

SymPDE $\rightarrow$ Model

Incorporating PDE symmetries into equivariant models

MSc Thesis intermediate presentation

Elias Dubbeldam



UNIVERSITY
OF AMSTERDAM

Supervised by Alex Gabel

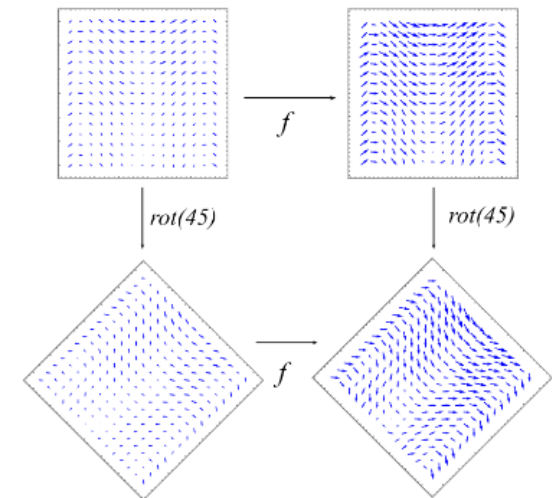
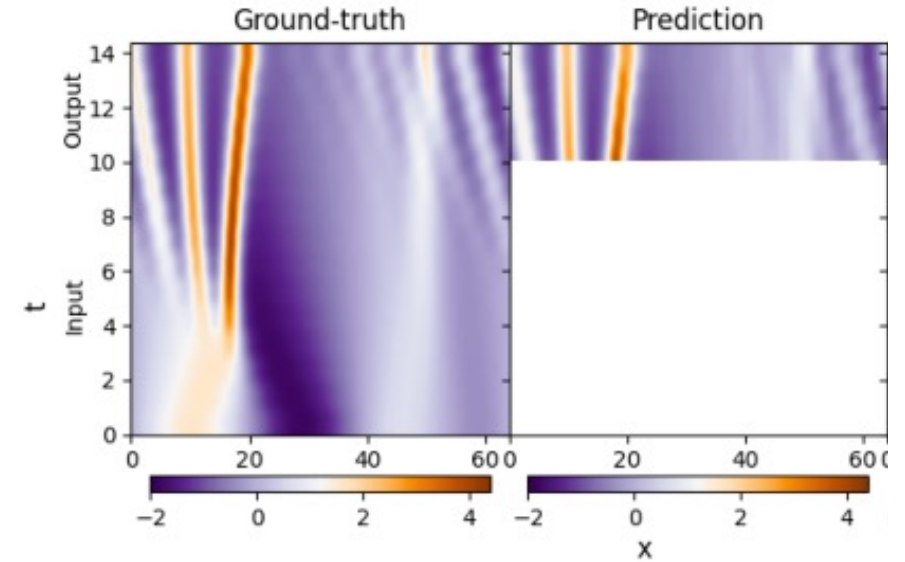
slides at: github.com/elidub/SymPDE

Outline

- Motivation
- Literature overview
- SymPDESuite
- Equivariant-learned layers
- Planning

Motivation

- Setting: PDE future prediction
- Neural PDE solvers can speed up classical solvers
- However:
Neural PDE solvers are data hungry
- Therefore:
Incorporate PDE symmetries into equivariant models: “SymPDE $\rightarrow$ Model”



Limitations of current equivariant neural PDE solvers

Data: Existing work analyses only a few PDEs, landscape is scattered.

- Models are hard to compare
- Contribution per symmetry is poorly understood

	Reference	Heat	Burger's	KdV	KS	NS	SWE	Other
Intrinsic equivariant	Single Symmetry G-CNN (Wang et al., 2021)	2D						Ocean currents, smoke convection
	Group equivariant FNO (Helwig et al., 2023)					2D, 3D	2D, 3D	
Extrinsic equivariant	Data augmentation (Brandstetter et al., 2022)		1D	1D	1D			
	Physics-informed NNs (Akhound-Sadegh et al., 2023)	1D	1D					
	Self-supervised (Mialon et al., 2023)		1D	1D	1D	2D		

Model

- Intrinsic models don't use all symmetries and only work discretized → Not all inductive biases are used
- Extrinsic models do not exploit symmetry inside model → No equivariance guarantees

Data: SymPDESuite

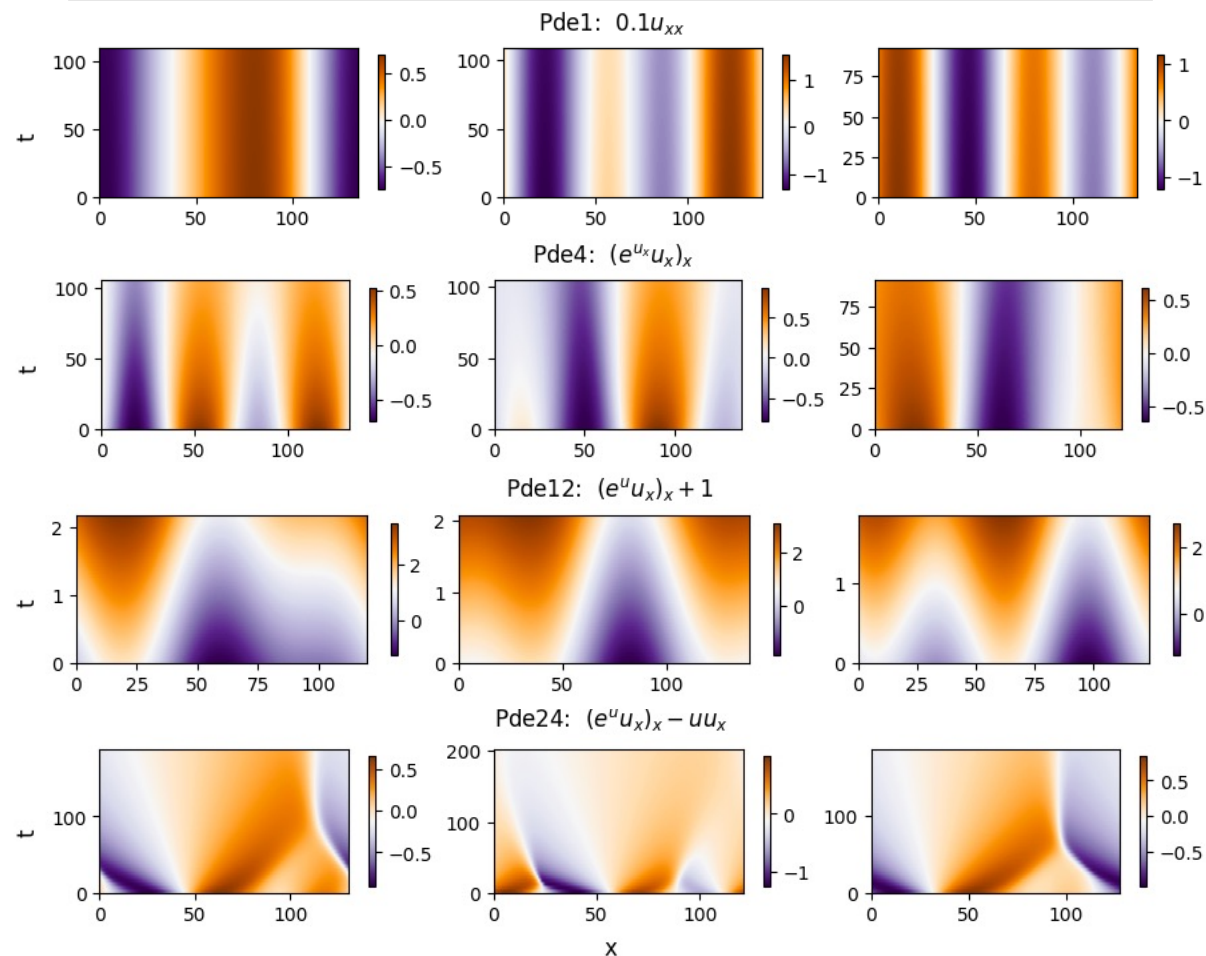
SymPDESuite: a benchmark for equivariant neural PDE solvers

- As a start, use the PDEs from Gabel, 2024.
- Most PDEs have much more symmetries than ‘standard’ PDEs

This solves the problem that current landscape is scattered, enabling to:

- Have a comparative overview equivariant neural PDE solvers
- Analyse and understand the contribution per type of symmetry

Index	PDE	Generators
1	$u_t = 0.1u_{xx}$	$X_1 = \partial_t, X_2 = \partial_x, X_3 = 2t\partial_t + x\partial_x, \dots, X_\infty = b(t, x)\frac{\partial}{\partial u}$
4	$u_t = (e^u u_x)_x$	$X_1 = \partial_t, X_2 = \partial_x, X_3 = 2t\partial_t + x\partial_x, \dots, X_7 = \partial_x - t\partial_t + \partial_u$
12	$u_t = (e^u u_x)_x + 1$	$X_1 = \partial_t, X_2 = \partial_x, X_3 = e^{-t}\frac{\partial}{\partial t} + e^{-t}\frac{\partial}{\partial u}, X_4 = x\frac{\partial}{\partial x} + 2\frac{\partial}{\partial u}$
24	$(e^u u_x)_x - uu_x$	$X_1 = \partial_t, X_2 = \partial_x, X_3 = t\frac{\partial}{\partial t} + (x+t)\frac{\partial}{\partial x} + \frac{\partial}{\partial u}$

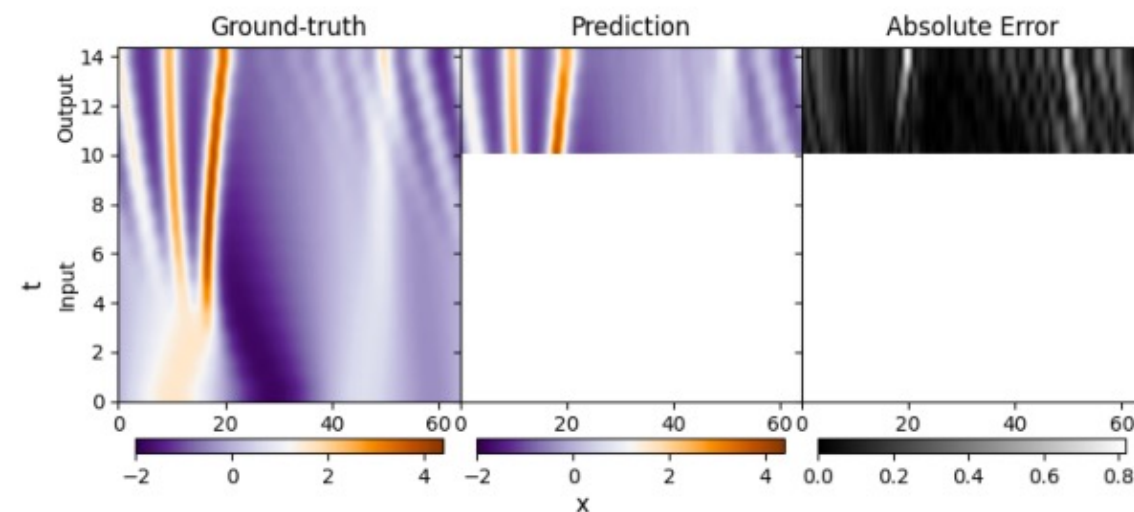


Experimenting baseline models

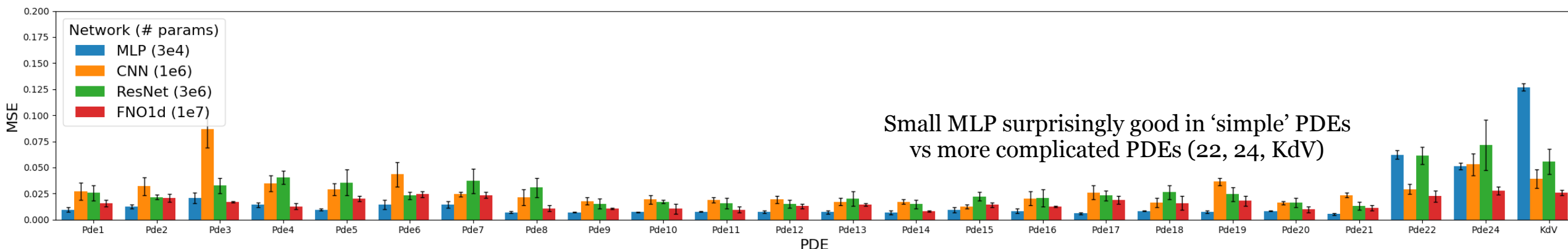
Task

- PDE future prediction on SymPDESuite
- $(N_{train}, N_{val}, N_{test})=(500,100,100)$
- ✓ Have a comparative overview of SymPDE → Model techniques

Task example (KdV)



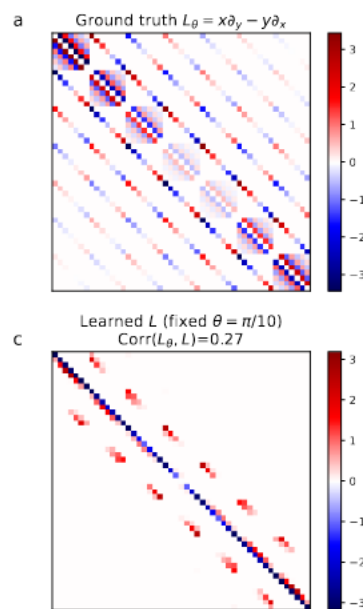
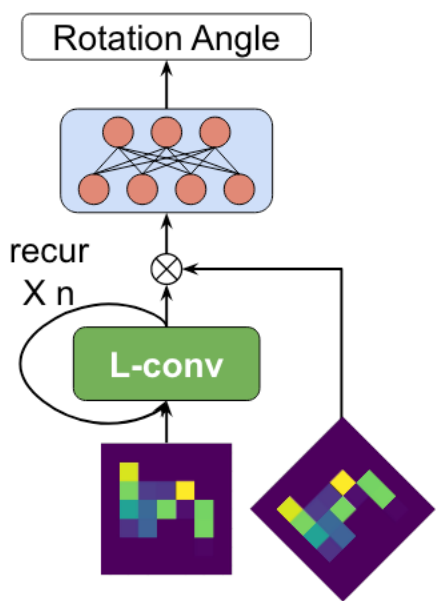
Performance across models and PDEs



Equivariant Model: Literature

- G-CNNs not feasible due to the number of symmetries in PDEs
- What about learning an intrinsic equivariant architecture?

L-Conv Dehmamy *et al* (NeurIPS21)

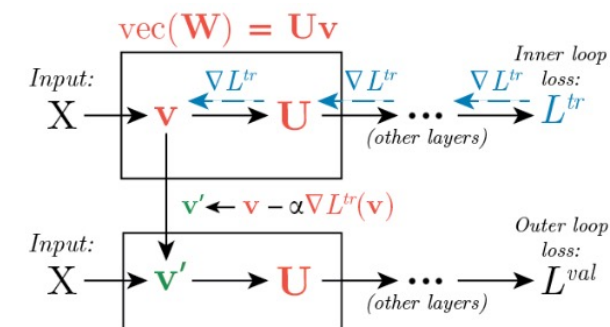


Meta-learning symmetries by reparameterization

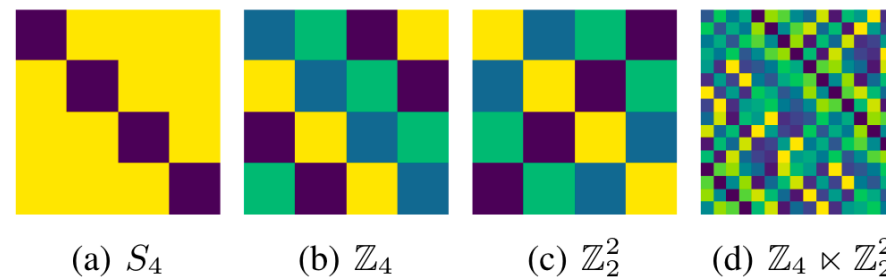
Zhou, Knowles & Finn (ICRL21)

$$\begin{pmatrix} \pi(e) \\ \pi(g) \end{pmatrix} \rightarrow \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 0 & 1 \\ 1 & 0 \end{pmatrix} \underbrace{\begin{pmatrix} \text{red} \\ \text{blue} \end{pmatrix}}_{\text{parameters}} = \begin{pmatrix} \text{red} \\ \text{blue} \\ \text{blue} \\ \text{red} \end{pmatrix} \xrightarrow{\text{reshape}} \begin{pmatrix} \text{red} & \text{blue} \\ \text{blue} & \text{red} \end{pmatrix} \underbrace{\hspace{1cm}}_{\text{layer weights}}$$

$U \bullet v = \text{vec}(W) \rightarrow W$



Equivariant MLP Finzi, Welling & Wilson (PLMR21)

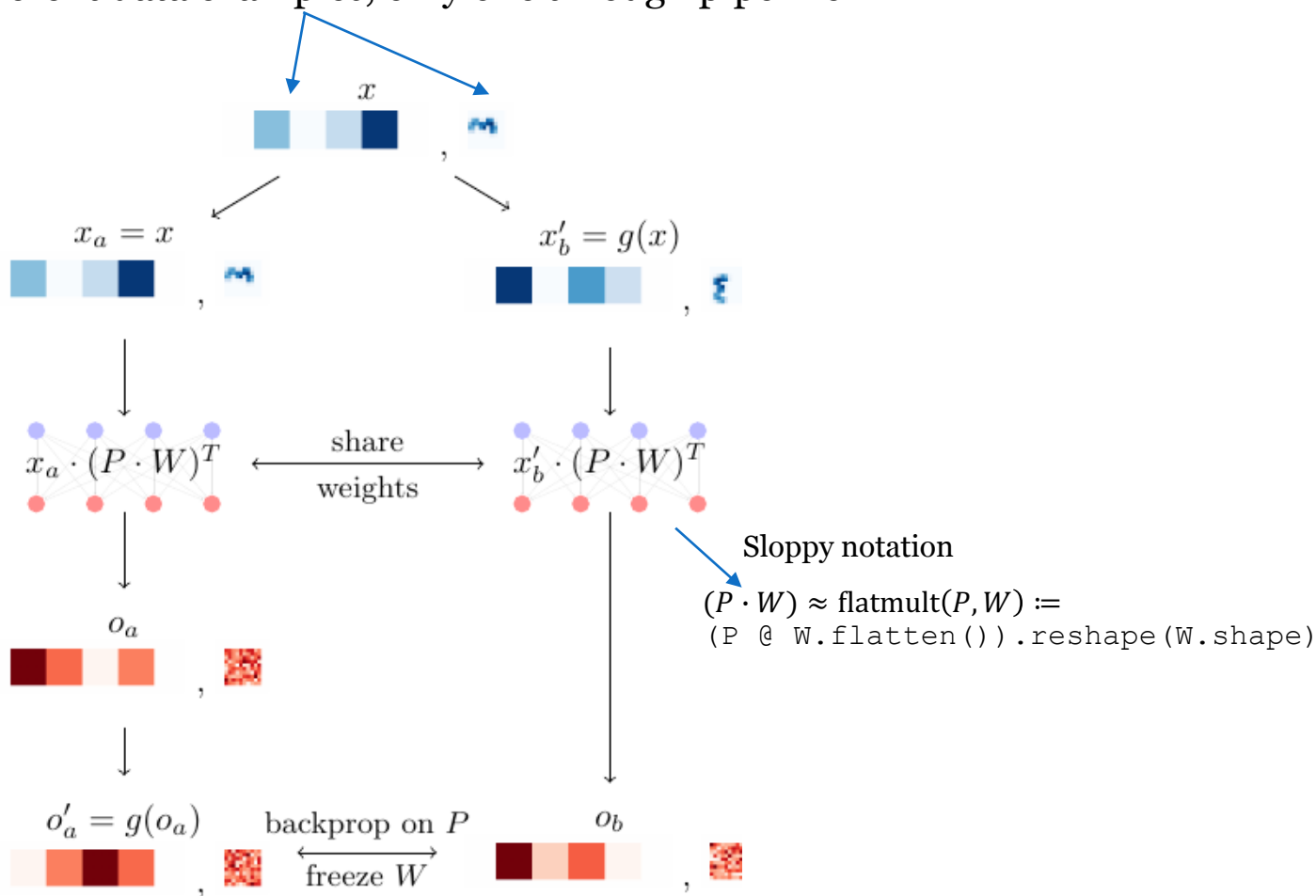


Equivariant Learned Layers

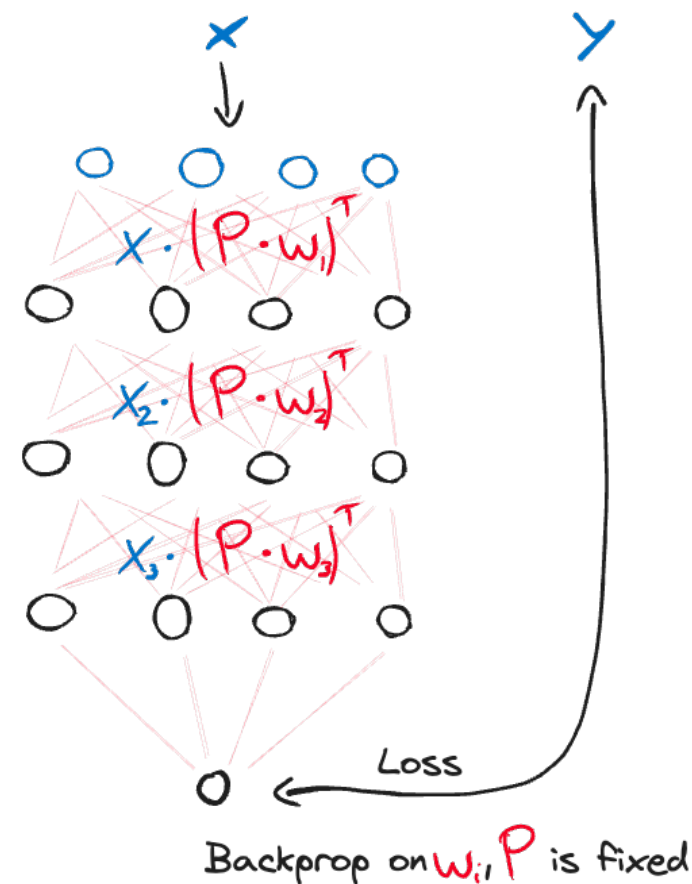
Learn an equivariant weight mapping P in step A, and use in a downstream task in step B

Step A

Two different data examples, only one through pipeline



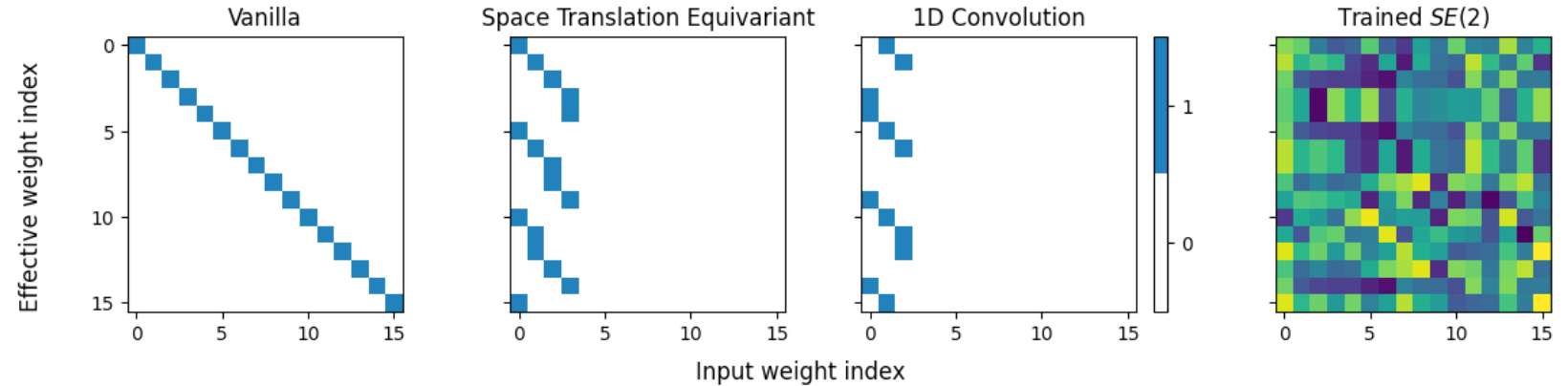
Step B



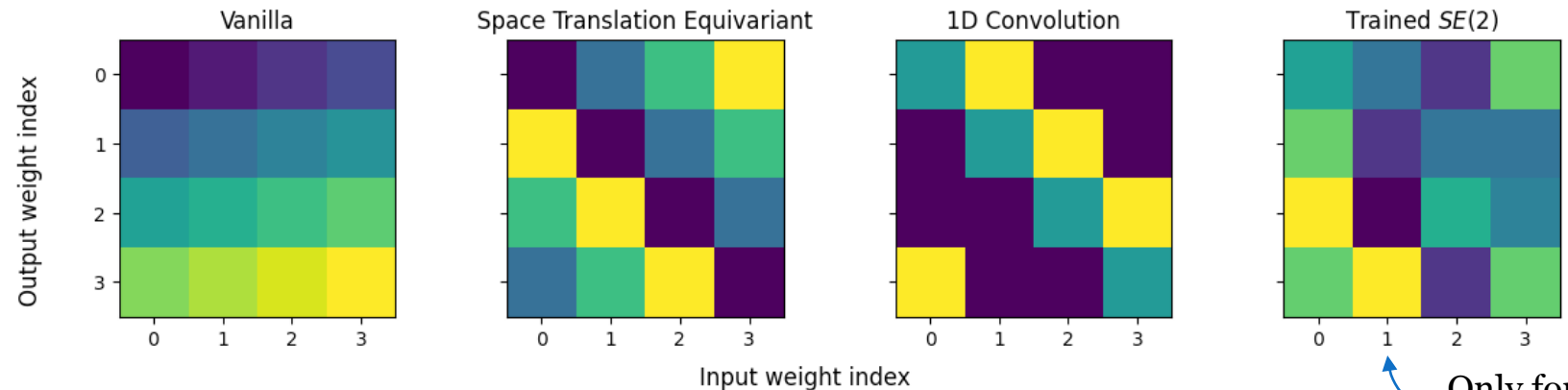
Weight mapping P

- Simple symmetries can be written down

Weight mapping P



Effective weight mapping ($P \cdot W$)

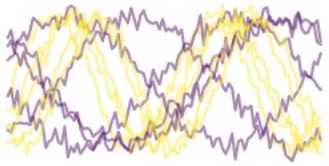


Only for visual purposes

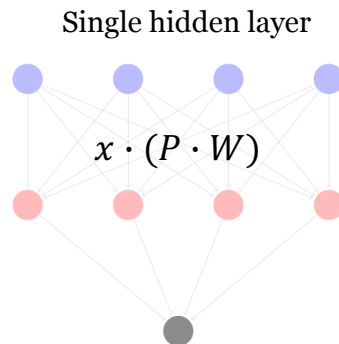
Preliminary results

Synthetic dataset **Sine1d**

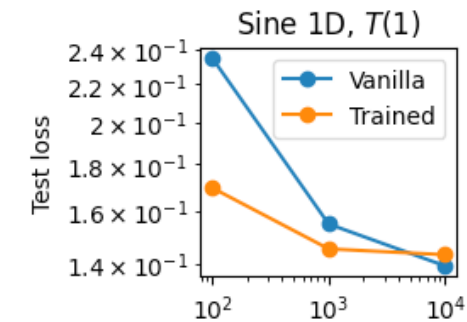
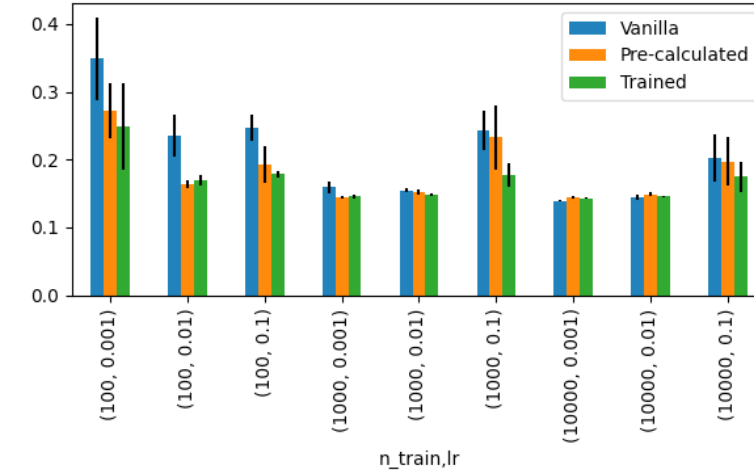
Show data To understand how what the data is



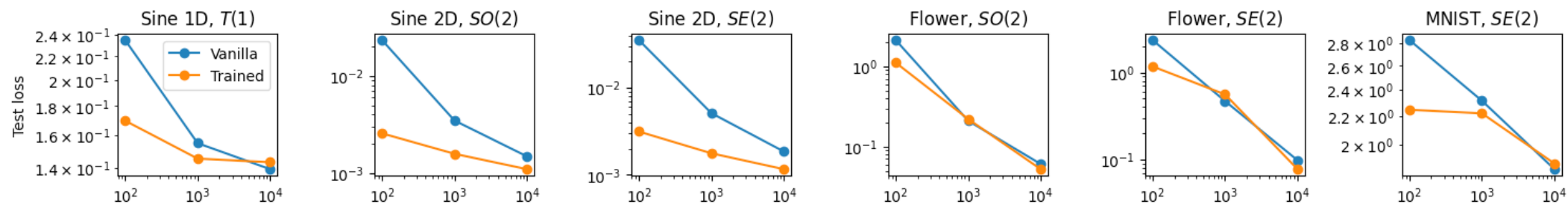
Input data What is presented to the model



Varying train size and LR

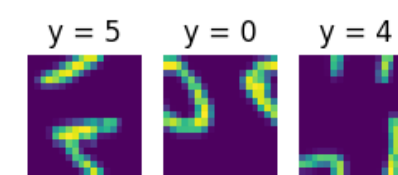
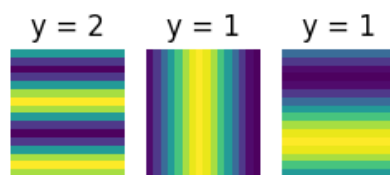
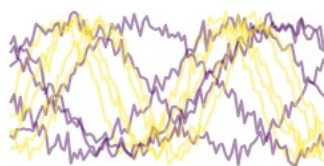


Preliminary results

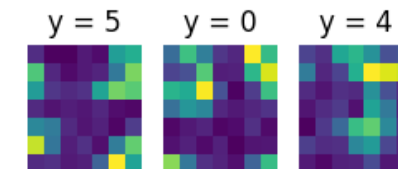
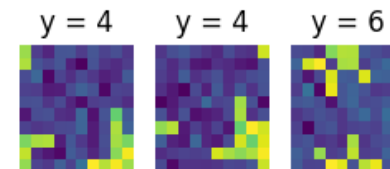
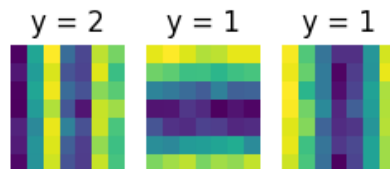


Train size

Show data To understand how what the data is



Input data What is presented to the model



Equivariant Learned Layers

Also experimented with

- ✓ Noise as input for step A
- ✓ Deeper networks
- ✓ Multi-target regression

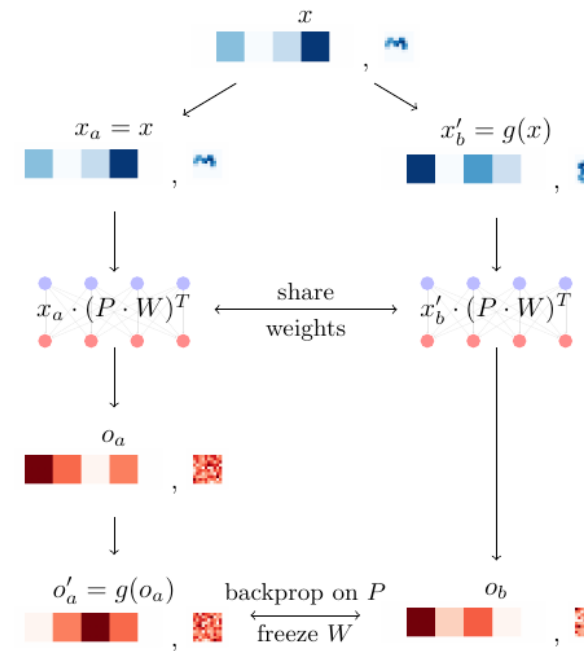
Would like to do

- Learn a matrix decomposition of P
- Weight mappings for conv kernels instead of linear layers
- Implicit kernel instead of $\text{flatmult}(P, W)$

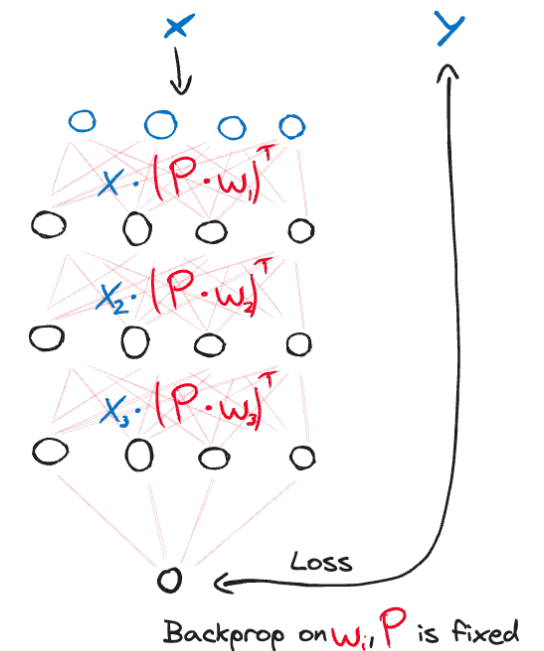
Will do

- Compare against equivariant or PDE literature?

Step A



Step B

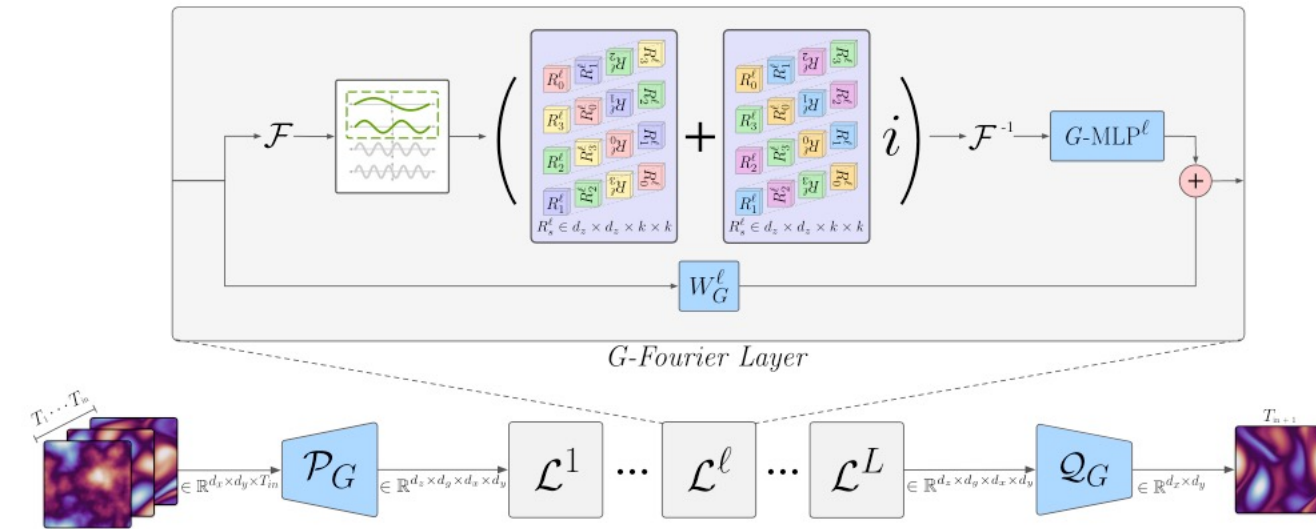


Planning

- ❑ March Further development of method
- ❑ April Implementing new method and evaluate baselines
- ❑ May-June Finalizing and writing thesis

Backup slides

G-FNO



		NS
	# PAR. (M)	TEST (%)
FNO	0.93	8.41(0.41)
FNO+p4	0.93	10.44(0.47)
FNO+p4m	0.93	22.09(1.46)
G-FNO-p4	0.85	4.78 (0.39)
G-FNO-p4m	0.84	<u>6.19</u> (0.61)
U-NET-p4	3.65	18.40(0.44)
RADIALFNO-p4	1.03	9.21(0.26)
RADIALFNO-p4m	0.95	10.86(0.18)

LPS in PINNs

- Incorporate symmetry loss term using MLPs in PINN loss:

$$\mathcal{L}(\theta) = \alpha \mathcal{L}_{\text{PDE}} + \beta \mathcal{L}_{\text{data-fit}} + \gamma \mathcal{L}_{\text{sym}}$$

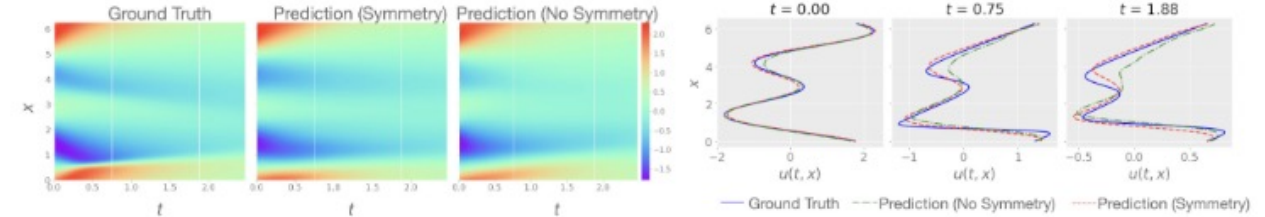
- $\mathcal{L}_{\text{PDE}}$ learns solution solving the PDE; $\mathcal{L}_{\text{data-fit}}$ learns IC and BC; $\mathcal{L}_{\text{sym}}$ learns LPS.
- $\mathcal{L}_{\text{sym}} = \sum_{k=1}^K J_{\Delta}^{\top} \text{coef}(\text{pr}^{(n)} \mathbf{v}_k)$
 - The orthogonality of the K prolonged vector fields and the gradient vector.
 - Equivalent to $\text{pr}^{(n)} \mathbf{v}[\Delta] = 0$ when $\Delta = 0$. That is: asserting that applying a generator (e.g. $\mathbf{v} = 2\nu t \partial_x - xu \partial_u$) to the PDE (e.g. $\Delta = u_t - \nu u_{xx}$) still solves the PDE.

Table 1: The average test set mean-squared error for the Heat equation.

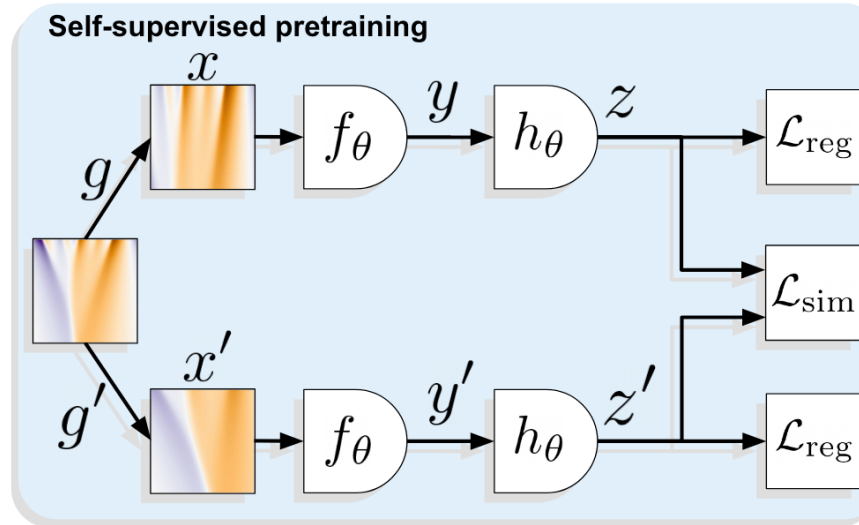
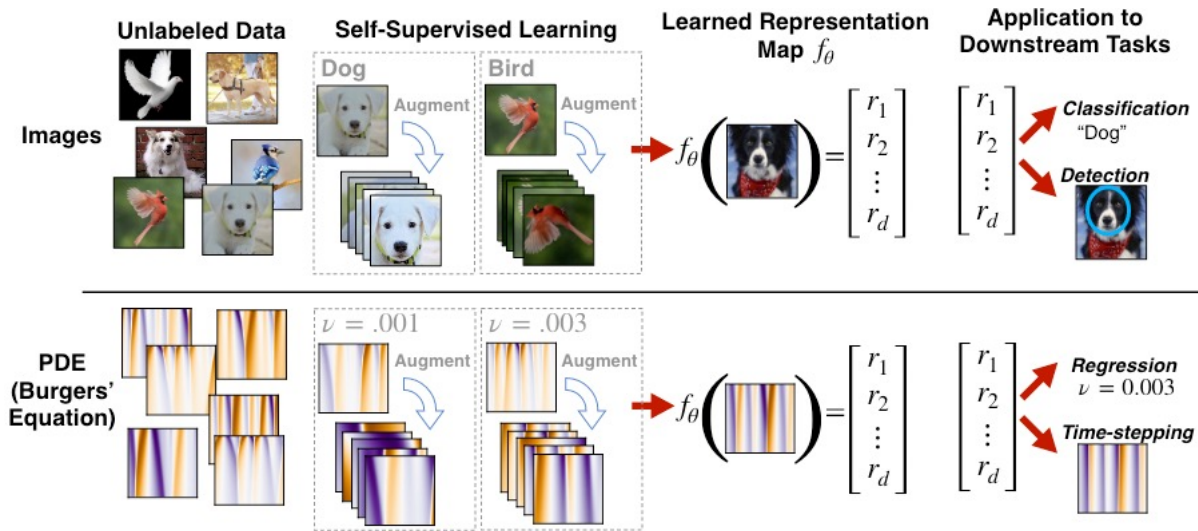
Number of Points (N_r)	No Symmetry	Symmetry
500	1.12 ± 0.58	0.30 ± 0.15
2000	0.36 ± 0.19	0.24 ± 0.14
10000	0.22 ± 0.14	0.21 ± 0.13

Table 2: The average test set mean-squared error for Burgers' equation.

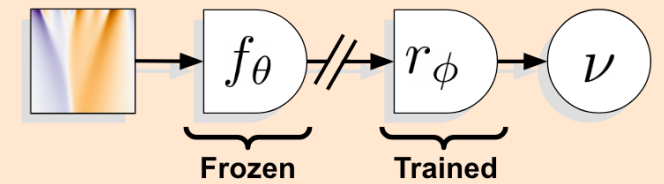
Number of Points (N_r)	No Symmetry	Symmetry
5000	0.041 ± 0.042	0.034 ± 0.039
25000	0.030 ± 0.038	0.017 ± 0.020
100000	0.018 ± 0.022	0.013 ± 0.020



SSL



Supervised downstream task



Representation conditioned time-stepping

